

KMP算法讲义

1.最简单的是暴力算法

暴力算法就是将两个字符串依次匹配。

```
1  #include<stdio.h>
2  #include<string.h>
3
4  /*暴力*/
5  int main(){
6      char source[] ="This is data structure,please find data"; //主
    字符串
7      char *find = "data";          // 模式字符串
8
9      int where = 1;
10     int lenSource = strlen(source);
11     int lenFind = strlen(find);
12     int i = 0 ,j = 0;
13     for(;i <= lenSource - lenFind; i++){
14         for(j = 0;j < lenFind; j++){
15             if(source[i+j] != find[j]){
16                 break;
17             }
18         }
19         if(j == lenFind){
20             printf("第%d个匹配的位置: %d\n",where,i);
21             where++;
22         }
23     }
24
25     return 0;
26 }
```

注

我前面也写了一个关于KMP算法的文档，第二个for之前，增加首字符匹配。如下：

```
1  for(;i <= lenSource-lenFind; i++){
2      if(source[i] != find[0]){
3          continue; //找到首字母相同，再开始匹配
4      }
5      for(j = 0;j<lenFind;j++){
6          if(source[i+j] != find[j]){
7              break;
```

```
8         }
9     }
10    if(j == lenFind ){
11        printf("第%d个匹配的位置: %d\n",where,i);
12        where++;
13    }
14 }
```

仔细分析发现，这是多余的。因为第二个for其实也是首字符匹配，若匹配不成功就break了。

但是有一点是肯定的，字符串匹配时，**首字符**是最关键的，这也是KMP算法的一个关键点。

2.KMP算法

这个算法由[高德纳](#)和[沃恩·普拉特](#)在1974年构思，同年[詹姆斯·H·莫里斯](#)也独立地设计出该算法，最终三人于1977年联合发表。

它由两个要点：

- i不回退。因为一旦回退就意味时间复杂度的提高，万一模式字符串非常长——这种情况在大数据场景下是很常见的，就浪费了。
- 时间复杂度为M+N，不能比这个还高。实际上，这个复杂度就意味着i是不能回退的。

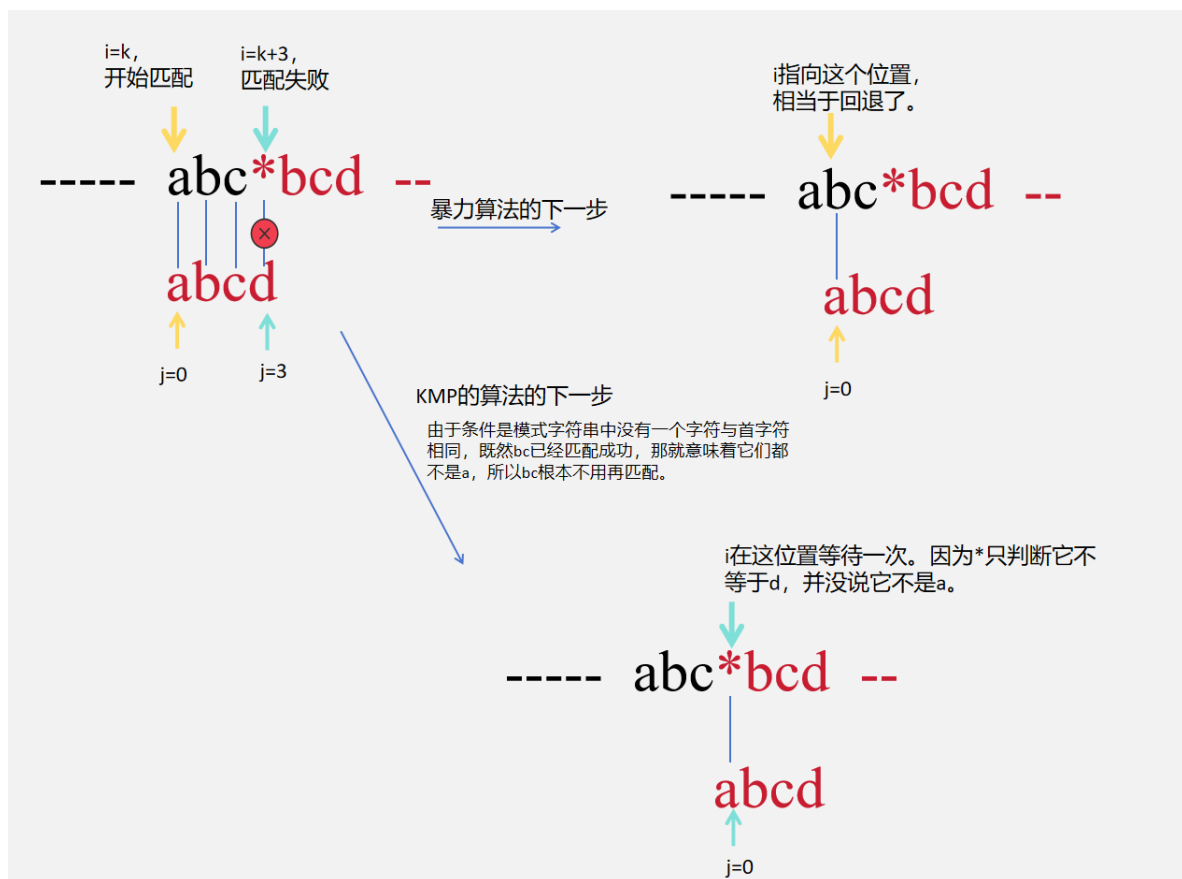
在暴力算法中，主字符串source中的i表面上看也没回退，但实际上，是回退的。因为遍历的是 `i+j`。

1. 假设 `i=4, j=3`，现在遍历的是 `source[7]`。
2. 假设 `source[7] != find[3]`，则 `j=0`，进入下一个循环 `i=5`，现在遍历的是 `source[5]`——回退了。

我们分两步来讲解KMP算法

①假设某个字符串中，不存在任何一个字符与首字符相同。

举个例子，如下图。



实现代码如下：

```

1  #include<stdio.h>
2  #include<string.h>
3
4  /*简单优化*/
5  int main(){
6      char source[] ="This is abcd find abcabcd,please find ";
7      char *find = "abcd";
8      /**
9       * 1.找出首字母相同的字母
10      * 2.然后往下匹配
11      * 3.匹配成功就划走
12      * 4.匹配不成功，就回退.回退到不匹配的index-1
13      */
14      int where = 1;
15      int lenSource = strlen(source);
16      int lenFind = strlen(find);
17      int i = 0, j = 0;
18
19      for(;i < lenSource-lenFind; i++){
20
21          /**现在字符不相等要分为两种情况：
22           * 1.首字母不相同，此时j=0，相当于模式字符串还没有真正开始匹配
23           * 2.模式字符串中间的字符不相等，j此时不是0了。
24          */

```

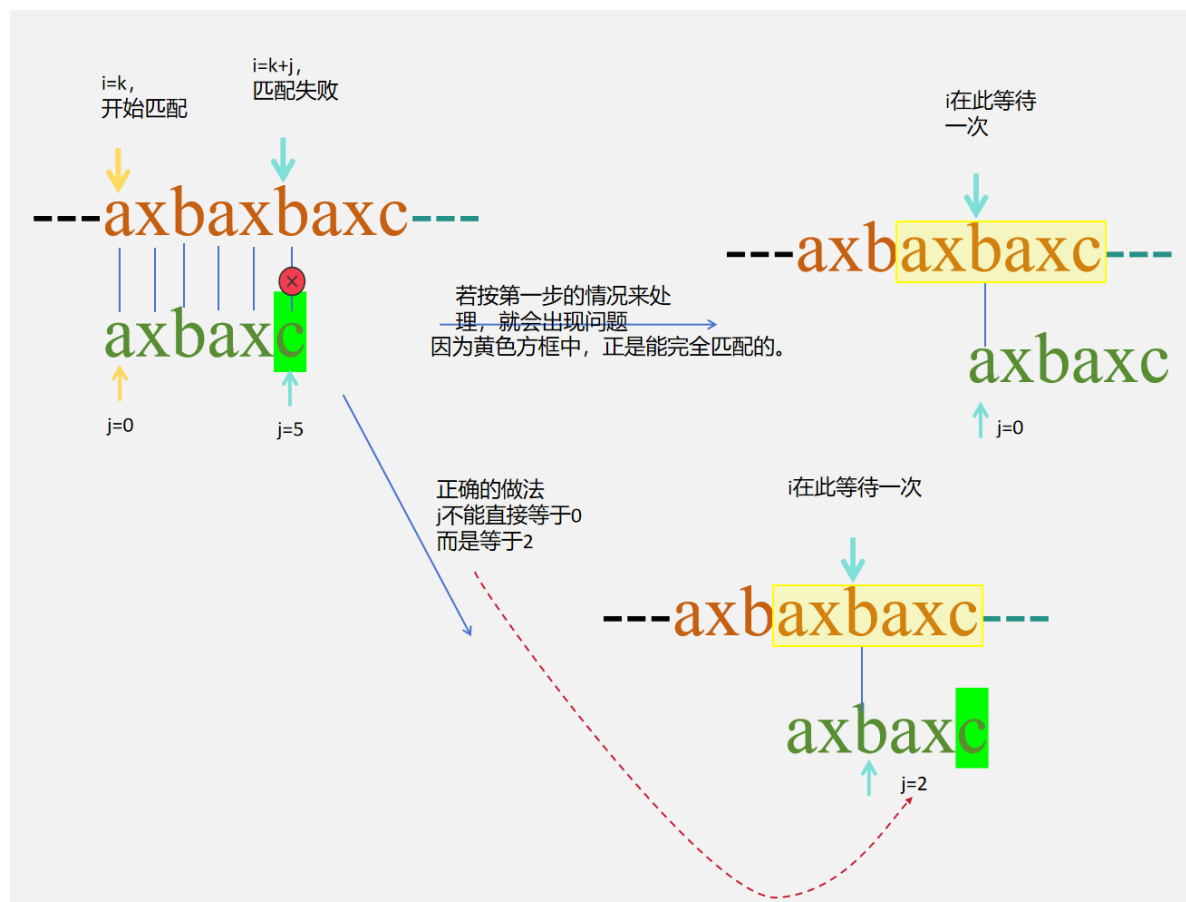
```

25     if(source[i] != find[j]){
26         i = (j > 0) ? i - 1 : i;
27         /* 手指
28         j>0,表示两个字符不相等发生在中间,此时i要等待一次,
29         所以i = i-1, 下一个循环, i又++了。
30         */
31         j = 0; //这个j始终归零
32     }else{
33         // 这是两个字符串中的字符相同,首字母相同也包含在内。
34         j++;
35     }
36
37     if(j == lenFind ){
38         printf("第%d个匹配的位置: %d\n",where,i-lenFind);
39         where++;
40     }
41 }
42 }

```

②假设某个字符串中, 存在与首字母相同的字母。

先举个例子, 看下图:



关于上面的案例 $j=2$, 带来了两个问题。

- `j=2` 从何而来？这是模式字符串所隐含的信息，字符串在c匹配错误，而c前面两个字符（ax），正好与首字符串（ax）相同。`j=2`，表达的就是这个ax的长度。
- `j=2`，意味着c前面的ax没有和主字符串进行比较。这样做合适吗？完全合适。因为主字符串的ax在上一次的匹配过程中是成功的，在下一次匹配过程时，与它相比较的正是模式字符串的头——也是两个字符的长度。

总之，很多关键信息都藏在这个2中，实际上，在第一步中，我们将 `j=0` 并不是错误，而是模式串abcd没有一个与首字母相同的字符，本身就是0。

问题来了，像 `j=2`，`j=0`，这些信息该如何获取的。KMP算法准备了next数组。我们先来说说数字的值。如下图：

abce	axbaxc	abcabcbcd
0000	000120	0001234560

它的计算方式：

1. next数组与模式串长度完全相等，一个字符对应next数组中的一个数字。
2. 首字符对应的数字为0，即`next[0] = 0`。
3. 从第二个字符开始往后找，找到与前缀相同的最大长度。我们以2~3两个字符串为例。

	axbaxc	
next:	0	首数值为0
next:	00	x与首字符串不同
next:	000	b与首字符串不同
next:	0001	a与首字符串不同
next:	00012	这是的x不能与a比较，而是与a后面的x比较
next:	000120	这是的c不能与a比较，而是与x后面的b比较

abcababcd

next: 0

next: 00

next: 000

next: 0001

next: 00012

next: 000123

next: 0001234

这是的a不是与首字符比较，而是与第二a比较为什么？

next数组实现如下：

```

1 void calNext(int *next, char *find, int length){
2
3     if(next == NULL || find == NULL)
4         return;
5
6     int i = 1; // 从1开始,
7     for(; i < length; i++){
8         if(find[i] == find[next[i-1]]){
9             next[i] = next[i-1] + 1;
10        }
11    }
12 }

```

最终实现代码如下。我用for实现的，网上有更简洁的代码，用while实现的。但道理都差不多。

```

1 int main(){
2     char source[] = "xyzaabaabaacd";
3     char *find = "aabaac";
4     int where[100] = {0};
5     int whereIndex = 0;
6     int lenSource = strlen(source);
7     int lenFind = strlen(find);
8     int *next = (int*) malloc(lenFind * sizeof(int));
9     memset(next, 0, lenFind * sizeof(int));

```

```

10     calNext(next, find, lenFind);    // 获得next数组
11
12
13     int i = 0, j = 0 ;
14
15     for(;i < lenSource;i++){
16         if(source[i] != find[j]){
17             i = (j > 0) ? i - 1 : i;    // 此处i是等待，不是回退，若用
while可消除此代码
18             j = (j > 0) ? next[ j-1 ] : 0;
19             // 案例1时，j直接等于0，本是上它next数组的的值为0。
20             // 案例2,3中，j不能再直接被赋值为0，是next数组决定的。
21         }else{
22             j++;
23         }
24
25         /**这里做了一点改变，即将匹配到的位置保存了起来。*/
26         if(j == lenFind){
27             where[whereIndex] = i - lenFind + 1;
28             whereIndex++ ;
29             j = 0;
30         }
31     }
32     for(i = 0; i < whereIndex;i++ ){
33         printf("第%d个匹配的位置:%d \n",i,where[i]);
34     }
35
36     free(next);
37     return 0;
38 }

```

总结与心得

记得给423上Java课时，也讲了KMP算法，当时用Java来实现的，讲成什么样，不记得了，但记得用Java写的代码比上面的要简练——道理是一样的。但我不愿意再翻出来了，宁愿再写一次。对于初学者，千万不要以为，代码你看懂了就行，而是认真写出来，踏踏实实地锻炼自己的编程思维，后面再学会使用AI，肯定有一碗饭吃的。